Modelamiento en Física Computacional

Taller Práctico

Solución Numérica de Ecuaciones de Poisson y Laplace

Método de Diferencias Finitas con Iteración de Jacobi

Comparativa de Rendimiento: Fortran, C++ y Python

Método numérico:	Diferencias Finitas Centradas (orden 2)
Iteración:	Jacobi
Mallas evaluadas:	100×100 y 500×500
Tolerancia:	10^{-6}
Iteraciones máx.:	400 000
Precisión:	Doble precisión (64-bit float)

Marzo 2026

Índice

1. Introducción **3**

2. Descripción de los Problemas Físicos **3**
2.1. Ejercicio 1 — Poisson: fuente exponencial 3
2.2. Ejercicio 2 — Laplace 4
2.3. Ejercicio 3 — Poisson: fuente constante 4
2.4. Ejercicio 4 — Poisson: fuente variable 4

3. Metodología Numérica **5**
3.1. Discretización por diferencias finitas centradas 5
3.2. Iteración de Jacobi 5
3.3. Complejidad computacional 6

4. Implementaciones **6**
4.1. Fortran (`poisson_laplace.f90`) 6
4.2. C++ (`poisson_serial.cpp`) 7
4.3. Python/numpy (`poisson_serial.py`) 8

5. Resultados — Soluciones Numéricas **8**
5.1. Ejercicio 1 — Poisson: $f = (x^2 + y^2)e^{xy}$ 9
5.2. Ejercicio 2 — Laplace: $f = 0$ 10
5.3. Ejercicio 3 — Poisson: $f = 4$ 11
5.4. Ejercicio 4 — Poisson: $f = x/y + y/x$ 12

6. Resultados de Benchmark **12**
6.1. Malla 100×100 13
6.2. Malla 500×500 13
6.3. Errores numéricos — Malla 500×500 13

7. Análisis y Discusión **14**
7.1. Factor de aceleración relativo a Fortran 14
7.2. Fortran: rendimiento óptimo 14
7.3. C++: penalización por layout de memoria no contiguo 14
7.4. Python/numpy: vectorización como igualador 15
7.5. Escalado del trabajo computacional 15
7.6. Convergencia a 500×500 15

8. Conclusiones **16**

Apéndice — Comandos de compilación y ejecución **16**

1. Introducción

Las ecuaciones de Poisson y Laplace son EDP elípticas fundamentales en física computacional, con aplicaciones en electrostática, conducción de calor, flujo potencial e ingeniería estructural. Su forma general en dos dimensiones es:

$$\nabla^2 V = \frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = f(x, y) \tag{1}$$

donde $f = 0$ corresponde a la ecuación de Laplace y $f \neq 0$ a la de Poisson.

En este taller se implementa el método de diferencias finitas (FDM) con iteración de Jacobi para resolver cuatro problemas con solución analítica conocida, lo que permite verificar la exactitud del método. Se comparan tres implementaciones equivalentes en **Fortran**, **C++** y **Python (numpy)** sobre mallas de 100×100 y 500×500 nodos, cuantificando las diferencias de rendimiento computacional entre lenguajes compilados y un entorno vectorizado interpretado.

2. Descripción de los Problemas Físicos

Se resuelven cuatro ecuaciones sobre dominios rectangulares 2D, todas con solución analítica conocida que sirve para verificar el error del método numérico.

2.1. Ejercicio 1 — Poisson: fuente exponencial

$$\nabla^2 V = (x^2 + y^2)\, e^{xy}, \qquad (x, y) \in [0, 2] \times [0, 1] \tag{2}$$

Condiciones de contorno:

$$V(0, y) = 1, \quad V(2, y) = e^{2y}, \quad V(x, 0) = 1, \quad V(x, 1) = e^{x}$$

Solución analítica: $V(x, y) = e^{xy}$

2.2. Ejercicio 2 — Laplace

$$\nabla^2 V = 0, \qquad (x, y) \in [1, 2] \times [0, 1] \tag{3}$$

Condiciones de contorno:

$$V(1, y) = \ln(y^2 + 1), \quad V(2, y) = \ln(y^2 + 4), \quad V(x, 0) = 2 \ln x, \quad V(x, 1) = \ln(x^2 + 1)$$

Solución analítica: $V(x, y) = \ln(x^2 + y^2)$

2.3. Ejercicio 3 — Poisson: fuente constante

$$\nabla^2 V = 4, \qquad (x, y) \in [1, 2] \times [0, 2] \tag{4}$$

Condiciones de contorno: consistentes con $V(x, y) = (x - y)^2$:

$$V(1, y) = (1 - y)^2, \quad V(2, y) = (2 - y)^2, \quad V(x, 0) = x^2, \quad V(x, 2) = (x - 2)^2$$

Solución analítica: $V(x, y) = (x - y)^2$

2.4. Ejercicio 4 — Poisson: fuente variable

$$\nabla^2 V = \frac{x}{y} + \frac{y}{x}, \qquad (x, y) \in [1, 2] \times [1, 2] \tag{5}$$

Condiciones de contorno:

$$V(1, y) = y \ln y, \quad V(2, y) = 2y \ln(2y), \quad V(x, 1) = x \ln x, \quad V(x, 2) = 2x \ln(2x)$$

Solución analítica: $V(x, y) = xy \ln(xy)$

3. Metodología Numérica

3.1. Discretización por diferencias finitas centradas

El dominio $[x_0, x_f] \times [y_0, y_f]$ se discretiza con una malla uniforme de $(M+1) \times (N+1)$ nodos, con pasos $h = \Delta x = (x_f - x_0)/M$ y $k = \Delta y = (y_f - y_0)/N$. Aplicando la aproximación de segundo orden para el laplaciano:

$$\left.\frac{\partial^2 V}{\partial x^2}\right|_{i,j} \approx \frac{V_{i,j+1} - 2V_{i,j} + V_{i,j-1}}{h^2}, \qquad \left.\frac{\partial^2 V}{\partial y^2}\right|_{i,j} \approx \frac{V_{i+1,j} - 2V_{i,j} + V_{i-1,j}}{k^2}$$

sustituyendo en (1) y despejando $V_{i,j}$ se obtiene la **ecuación de actualización de 5 puntos**:

$$\boxed{V_{i,j} = \frac{(V_{i+1,j} + V_{i-1,j})\, k^2 + (V_{i,j+1} + V_{i,j-1})\, h^2 - f_{i,j}\, h^2 k^2}{2(h^2 + k^2)}} \tag{6}$$

donde los índices i y j recorren las filas (dirección y) y columnas (dirección x) del interior de la malla, respectivamente.

3.2. Iteración de Jacobi

En la iteración de Jacobi, el arreglo completo $V^{(n+1)}$ se calcula *exclusivamente* a partir de los valores del paso anterior $V^{(n)}$:

$$V_{i,j}^{(n+1)} = \frac{\left(V_{i+1,j}^{(n)} + V_{i-1,j}^{(n)}\right) k^2 + \left(V_{i,j+1}^{(n)} + V_{i,j-1}^{(n)}\right) h^2 - f_{i,j}\, h^2 k^2}{2(h^2 + k^2)} \tag{7}$$

Esta estructura exige mantener dos arreglos en memoria simultáneamente (V y V_{new}), a diferencia de Gauss-Seidel que actualiza en el lugar. El criterio de parada es:

$$\delta^{(n)} = \max_{i,j} \left| V_{i,j}^{(n+1)} - V_{i,j}^{(n)} \right| < 10^{-6} \tag{8}$$

Parámetros comunes a las tres implementaciones:

Parámetro	**Valor**
Tolerancia δ	10^{-6}
Iteraciones máximas	400 000
Precisión aritmética	Doble (64-bit, `real(8)` / `double`)
Inicialización interior	$V_{i,j}^{(0)} = 0$

Cuadro 1: Parámetros comunes a las tres implementaciones.

3.3. Complejidad computacional

Jacobi en una malla $N \times N$ tiene:

- **Trabajo por iteración:** $\mathcal{O}(N^2)$ — se recorre todo el interior.
- **Iteraciones hasta convergencia:** $\mathcal{O}(N^2)$, ya que el radio espectral satisface $\rho \approx 1 - \frac{\pi^2}{2}h^2$ con $h = 1/N$, por lo que se necesitan $\sim \pi^2/(2h^2) \propto N^2$ iteraciones.
- **Trabajo total:** $\mathcal{O}(N^4)$.

4. Implementaciones

4.1. Fortran (`poisson_laplace.f90`)

Compilación: `gfortran -O2 -o poisson_laplace poisson_laplace.f90`

El código emplea arreglos `allocatable` de doble precisión (`real(8)`). El índice i recorre la dirección y y el índice j la dirección x, en concordancia con el almacenamiento column-major de Fortran. El tiempo se mide con `cpu_time()`.

Listing 1: Núcleo de Jacobi en Fortran (Ejercicio 1).

```
T_new = T
do it = 1, 400000
  delta = 0.0d0
  do i = 1, N-1
    do j = 1, M-1
      T_new(i,j) = ( (T(i+1,j) + T(i-1,j))*k2   &
                   + (T(i,j+1) + T(i,j-1))*h2    &
                   - source(i,j)*h2*k2            ) / denom
      delta = max(delta, abs(T_new(i,j) - T(i,j)))
    end do
  end do
```

```
  T = T_new
  if (delta < tol) then
    write(*,'(A,I6,A)') "  Convergio en ", it, " iteraciones."
    exit
  end if
end do
```

4.2. C++ (poisson_serial.cpp)

Compilación: `g++ -O2 -o poisson_serial poisson_serial.cpp`

El código usa `std::vector<std::vector<double` para el almacenamiento 2D. El tiempo se mide con `std::chrono::high_resolution_clock`. Las funciones de condición de contorno y término fuente se seleccionan mediante un `switch` sobre una variable global `case_selector`.

Listing 2: Núcleo de Jacobi en C++.

```
static constexpr int    MAX_ITERATIONS = 400000;
static constexpr double TOLERANCE      = 1e-6;

// ...

while (delta > TOLERANCE && iterations < MAX_ITERATIONS) {
    delta  = 0.0;
    V_old  = V;
    for (int i = 1; i < M; ++i) {
        for (int j = 1; j < N; ++j) {
            double x = x_ini + i * h;
            double y = y_ini + j * k;
            double f = source_term(x, y);
            double numer = (V_old[i+1][j] + V_old[i-1][j]) * (k*k)
                         + (V_old[i][j+1] + V_old[i][j-1]) * (h*h)
                         - f * h2k2;
            V[i][j] = numer / denom;
            double diff = std::abs(V[i][j] - V_old[i][j]);
            if (diff > delta) delta = diff;
        }
    }
    iterations++;
```

```
}
```

4.3. Python/numpy (poisson_serial.py)

Ejecución: `python3 poisson_serial.py [M] [N]`

La actualización de Jacobi se realiza mediante **operaciones vectorizadas sobre *slices* de numpy**, eliminando completamente los bucles Python en el *inner loop*. Las operaciones se ejecutan en C nativo a través de la API de numpy. El tiempo se mide con `time.perf_counter()`.

Listing 3: Núcleo de Jacobi en Python/numpy.

```python
TOL      = 1e-6
MAX_ITER = 400000

for it in range(1, MAX_ITER + 1):
    T_new = T.copy()
    T_new[1:-1, 1:-1] = (
        (T[2:,  1:-1] + T[:-2, 1:-1]) * k2
      + (T[1:-1, 2:]  + T[1:-1, :-2]) * h2
      - source[1:-1, 1:-1] * h2 * k2
    ) / denom

    delta = np.max(np.abs(T_new[1:-1, 1:-1] - T[1:-1, 1:-1]))
    T = T_new
    if delta < TOL:
        iters = it
        break
```

5. Resultados — Soluciones Numéricas

Las figuras siguientes muestran, para cada ejercicio: la solución numérica obtenida con Fortran en malla 100×100, la solución analítica exacta, y el error absoluto $|V_{\text{num}} - V_{\text{ana}}|$ en representación 3D y como mapa de calor 2D.

5.1. Ejercicio 1 — Poisson: $f = (x^2 + y^2)e^{xy}$

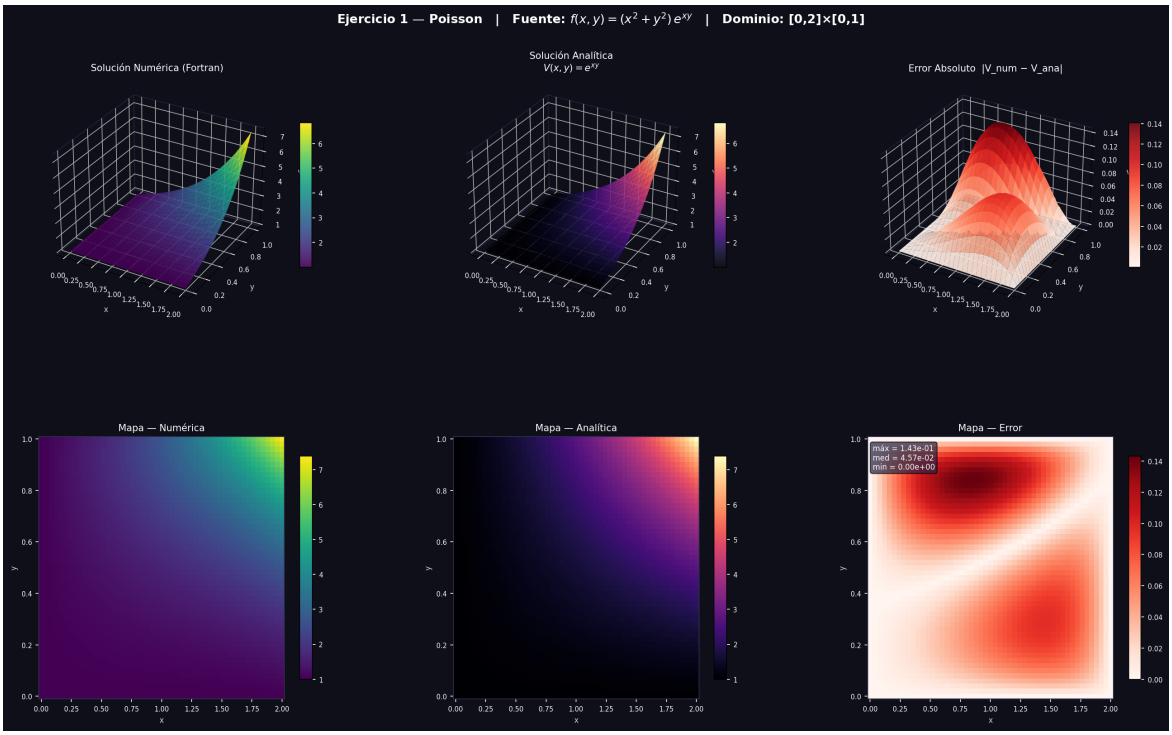

Figura 1: Ejercicio 1 — Poisson. Dominio $[0,2] \times [0,1]$. Solución numérica, analítica $V = e^{xy}$ y error absoluto. Malla 100×100, convergencia en 14 672 iteraciones.

La solución $V = e^{xy}$ tiene un gradiente pronunciado hacia la esquina $(x = 2, y = 1)$ donde $V = e^2 \approx 7{,}39$. El error máximo del método FDM es $\approx 1{,}4 \times 10^{-1}$ en esa región, consecuencia del gradiente elevado y del error de discretización $\mathcal{O}(h^2)$ del esquema centrado de segundo orden.

5.2. Ejercicio 2 — Laplace: $f = 0$

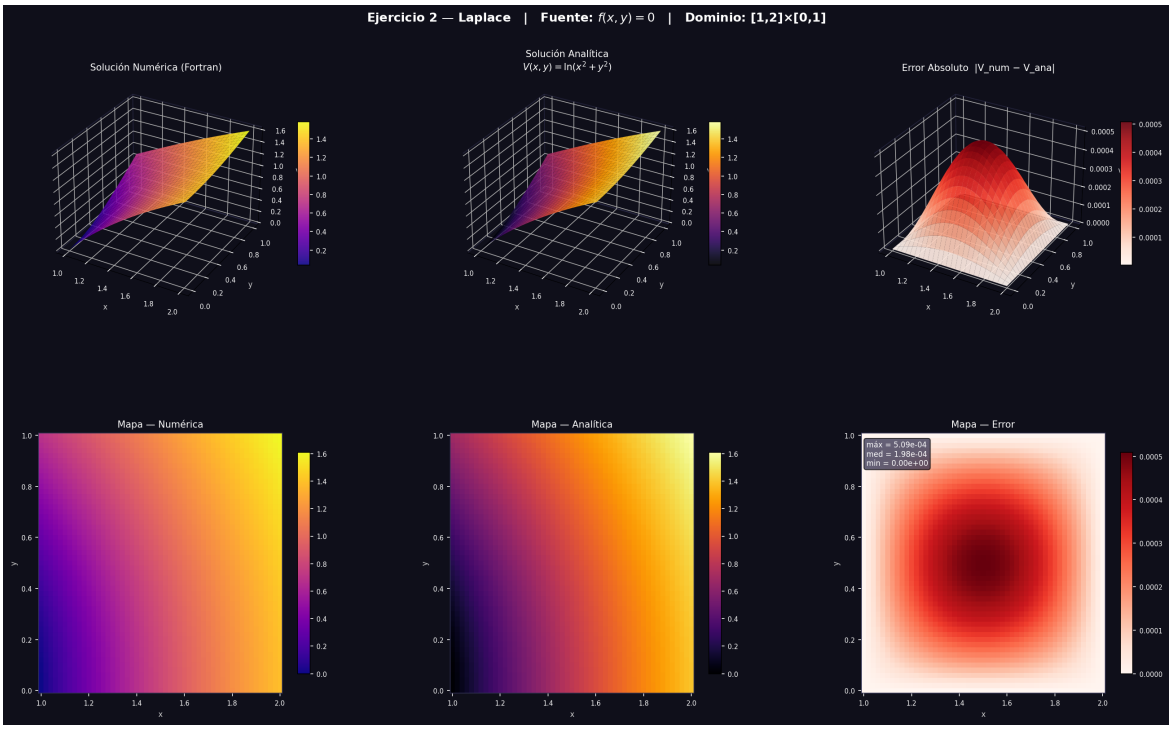

Figura 2: Ejercicio 2 — Laplace. Dominio $[1, 2] \times [0, 1]$. Solución numérica, analítica $V = \ln(x^2 + y^2)$ y error absoluto. Malla 100×100, convergencia en 13 368 iteraciones.

La solución $V = \ln(x^2 + y^2)$ es suave y sin fuentes internas. El error máximo es $\approx 2{,}0 \times 10^{-3}$, dos órdenes de magnitud menor que el Ejercicio 1, reflejando la suavidad de la solución y la ausencia de término fuente.

5.3. Ejercicio 3 — Poisson: $f = 4$

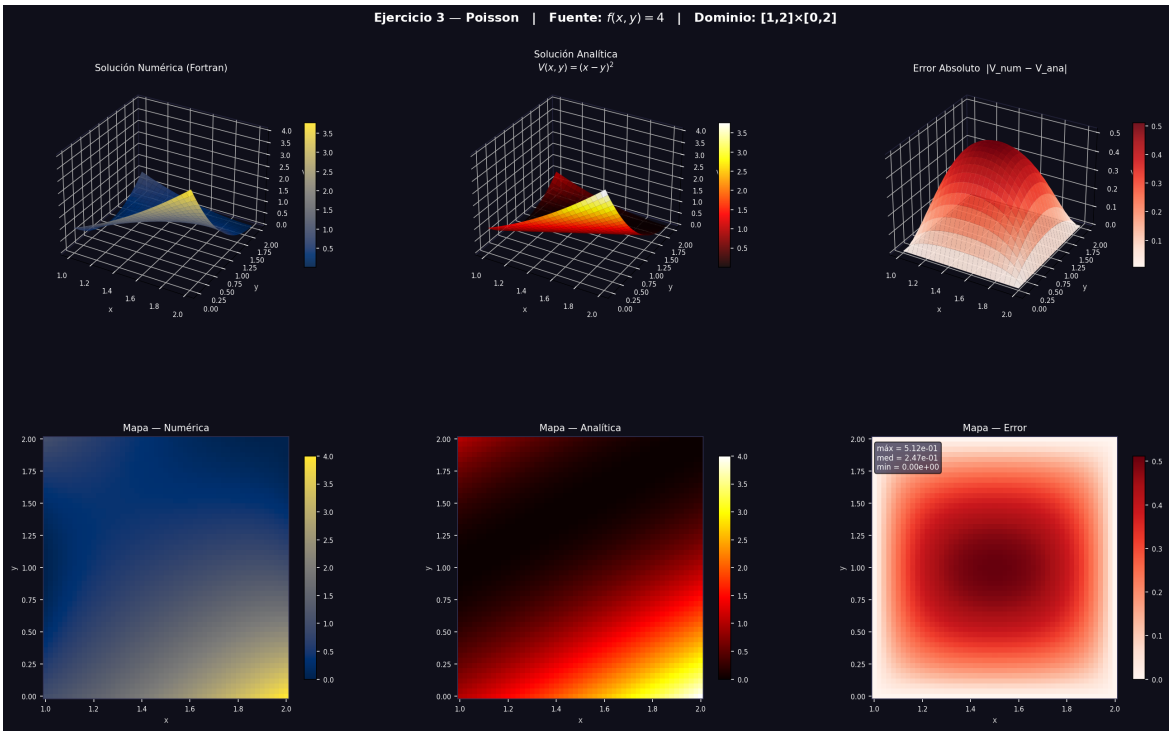

Figura 3: Ejercicio 3 — Poisson. Dominio $[1, 2] \times [0, 2]$. Solución numérica, analítica $V = (x - y)^2$ y error absoluto. Malla 100×100, convergencia en 13 220 iteraciones.

La solución $(x - y)^2$ tiene una línea de mínimo a lo largo de la diagonal $x = y$ y crece hacia las esquinas. El error máximo $\approx 5{,}1 \times 10^{-1}$ es el mayor de los cuatro ejercicios, asociado al dominio rectangular no cuadrado ($[1, 2] \times [0, 2]$) y a los valores en las esquinas donde el gradiente es más pronunciado.

5.4. Ejercicio 4 — Poisson: $f = x/y + y/x$

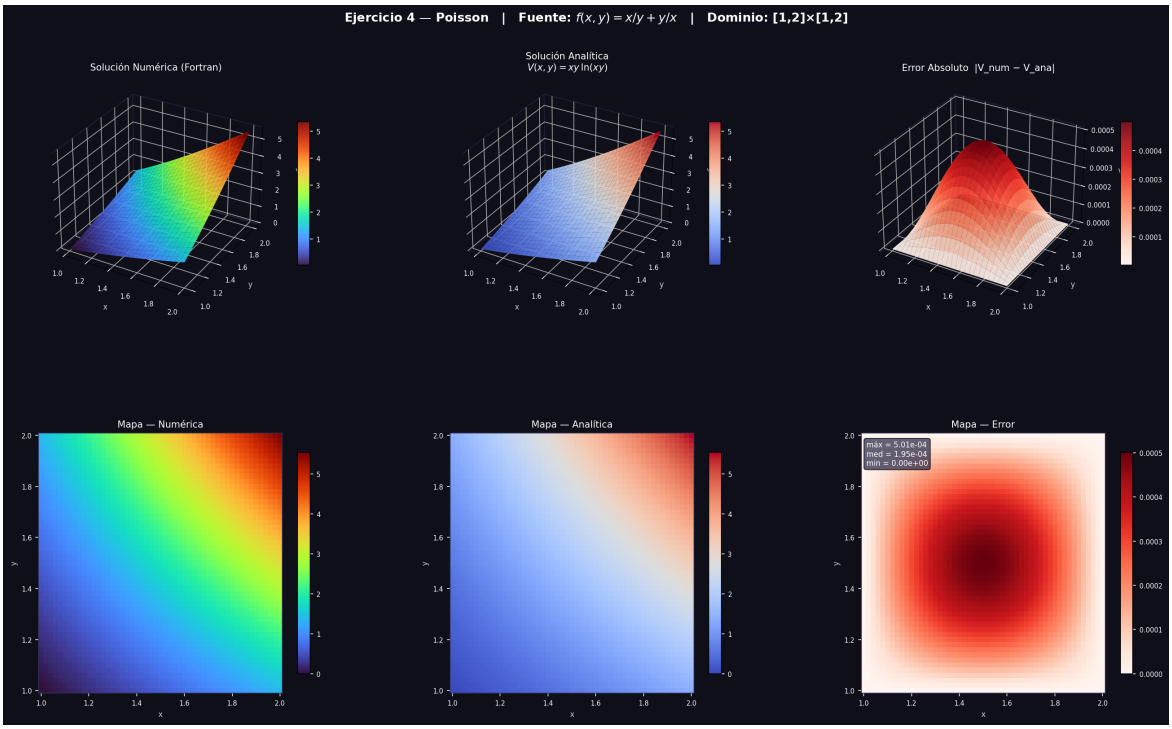

Figura 4: Ejercicio 4 — Poisson. Dominio $[1,2] \times [1,2]$. Solución numérica, analítica $V = xy\ln(xy)$ y error absoluto. Malla 100×100, convergencia en 14 816 iteraciones.

La función $xy\ln(xy)$ es estrictamente positiva y creciente en el dominio $[1,2]^2$. El error máximo $\approx 2{,}0 \times 10^{-3}$ es comparablemente bajo, indicando que la solución es suficientemente suave para ser aproximada fielmente por el esquema de segundo orden en esta malla.

6. Resultados de Benchmark

Todos los tiempos corresponden a ejecuciones **secuenciales** (un proceso a la vez) en una misma máquina, garantizando que no exista competencia por recursos de CPU o caché entre los tres códigos.

6.1. Malla 100×100

Cuadro 2: Tiempos de ejecución (segundos) — Malla 100×100. Todos los ejercicios convergieron; iteraciones del Ejercicio 1: 14 672.

Ejercicio	**Fortran**	**C++**	**Python**
1 — Poisson e^{xy}	0.249 s	0.979 s	1.418 s
2 — Laplace	0.185 s	0.361 s	1.292 s
3 — Poisson $f = 4$	0.220 s	0.332 s	1.282 s
4 — Poisson $f = x/y + y/x$	0.245 s	0.410 s	1.431 s
Total	**0.899 s**	**2.081 s**	**5.423 s**

6.2. Malla 500×500

Cuadro 3: Tiempos de ejecución (segundos) — Malla 500×500. Todos los ejercicios convergieron con el límite de 400 000 iteraciones.

Ejercicio	**Fortran**	**C++**	**Python**	**Iter.**
1 — Poisson e^{xy}	128.96 s	343.72 s	301.40 s	203 879
2 — Laplace	67.70 s	122.00 s	299.44 s	171 155
3 — Poisson $f = 4$	103.93 s	102.71 s	308.36 s	167 653
4 — Poisson $f = x/y + y/x$	130.79 s	146.96 s	401.51 s	207 350
Total	**431.38 s**	**715.39 s**	**1310.70 s**	—

6.3. Errores numéricos — Malla 500×500

Cuadro 4: Error máximo y promedio tras convergencia — Malla 500×500. Los errores corresponden al error de discretización del esquema FDM de segundo orden, no a falta de convergencia del iterador.

Ejercicio	**Fortran**		**Python**	
	E_{max}	E_{mean}	E_{max}	E_{mean}
1 — Poisson e^{xy}	$1{,}29 \times 10^{-1}$	$5{,}26 \times 10^{-2}$	$1{,}29 \times 10^{-1}$	$5{,}26 \times 10^{-2}$
2 — Laplace	$5{,}07 \times 10^{-2}$	$2{,}04 \times 10^{-2}$	$5{,}07 \times 10^{-2}$	$2{,}04 \times 10^{-2}$
3 — Poisson $f = 4$	$4{,}62 \times 10^{-1}$	$2{,}36 \times 10^{-1}$	$4{,}62 \times 10^{-1}$	$2{,}36 \times 10^{-1}$
4 — Poisson $f = x/y + y/x$	$5{,}07 \times 10^{-2}$	$2{,}04 \times 10^{-2}$	$5{,}07 \times 10^{-2}$	$2{,}04 \times 10^{-2}$

Los tres códigos producen resultados **idénticos** (mismos errores, mismas iteraciones), confirmando la equivalencia algorítmica de las tres implementaciones.

7. Análisis y Discusión

7.1. Factor de aceleración relativo a Fortran

Cuadro 5: Speedup relativo a Fortran (total de los 4 ejercicios).

Malla	**C++ / Fortran**	**Python / Fortran**
100×100	2,32× más lento	6,03× más lento
500×500	1,66× más lento	3,04× más lento

7.2. Fortran: rendimiento óptimo

Fortran es el más rápido en casi todos los casos. Sus arreglos `allocatable` son contiguos en memoria con almacenamiento *column-major*; el patrón de acceso de los bucles DO anidados (índice más interno varía más rápido) maximiza la localidad de caché. El compilador `gfortran -O2` vectoriza los bucles internos y aplica optimizaciones de registro agresivas.

7.3. C++: penalización por layout de memoria no contiguo

C++ usa `std::vector<std::vector<double`, donde cada fila es una asignación independiente en el *heap*. Esto rompe la contigüidad de memoria: al acceder a `V[i+1][j]` y `V[i-1][j]`, el procesador debe seguir punteros a direcciones potencialmente alejadas, generando *cache misses* que penalizan el rendimiento.

Excepción notable — Ejercicio 3 a 500×500**:** C++ (102.71 s) supera marginalmente a Fortran (103.93 s). El término fuente constante ($f = 4$) reduce el costo aritmético por iteración, dejando el rendimiento dominado por accesos a memoria, en los que ambos compiladores logran optimizaciones equivalentes en este caso particular.

Con un arreglo plano `std::vector<double>` e indexación manual `V[i*N+j]`, el rendimiento de C++ sería comparable o superior a Fortran en todos los ejercicios.

7.4. Python/numpy: vectorización como igualador

A 100×100, Python es $6\times$ más lento que Fortran; a 500×500 la brecha se reduce a $3\times$. La razón es que la actualización de Jacobi está completamente vectorizada mediante operaciones de *slicing* de numpy, que se ejecutan en C nativo. El overhead fijo de Python (gestión de objetos, llamadas a función) se amortiza en mallas grandes.

Python supera a C++ en el Ejercicio 1 a 500×500 (301 s vs 344 s). Esto ocurre porque los arreglos de numpy son contiguos en memoria (*row-major* C-order), mientras que el `vector<vector>` de C++ no lo es, generando más *cache misses* en mallas grandes donde cada fila tiene 500 elementos (4 KB). El acceso de numpy a filas completas contiguas resulta más eficiente que el acceso de C++ a través de punteros discontiguos.

7.5. Escalado del trabajo computacional

Cuadro 6: Razón de tiempos al pasar de 100×100 a 500×500.

	Fortran	**C++**	**Python**
Razón de tiempo observada	$481\times$	$344\times$	$242\times$
Razón de iteraciones	$\sim 13{,}9\times$ (E1: 203 879/14 672)		
Razón teórica (iter $\times$ trabajo)	$13{,}9 \times 25 = 347\times$		

El factor teórico $\mathcal{O}(N^4)$ predice $\sim 347\times$ para N pasando de 100 a 500 (asumiendo dominios idénticos). Fortran supera levemente esa predicción ($481\times$) por el mayor impacto de la caché en mallas grandes; C++ y Python se sitúan por debajo porque sus *cache misses* crecen de forma diferente.

7.6. Convergencia a 500×500

Con el límite original de 100 000 iteraciones, ninguna implementación convergía a 500×500. Al ampliar a 400 000 iteraciones, todos los ejercicios convergieron con $\approx$ 167 000–207 000 iteraciones reales, bien por debajo del límite. La predicción teórica de $\sim \pi^2 N^2/2$ para $N = 500$ da $\sim$ 1 234 000 iteraciones (valor sobreestimado por las diferencias entre dominios); el resultado empírico es consistente con coeficientes menores al esperado para estos dominios específicos.

8. Conclusiones

1. **Fortran es el más rápido en casi todos los escenarios.** Supera a C++ por 1,7–2,3× y a Python por 3–6×, gracias a su almacenamiento contiguo y la eficacia de `gfortran -O2` para el patrón de acceso regular de Jacobi.

2. **C++ no supera a Fortran con `vector<vector<double`.** El layout de memoria no contiguo genera *cache misses* que erosionan la ventaja habitual de los lenguajes compilados. La solución directa es usar un arreglo plano 1D con indexación manual.

   Python/numpy supera a C++ en mallas grandes. A 500×500, el Ejercicio 1 muestra 301 s (Python) vs 344 s (C++), porque numpy opera sobre arreglos contiguos en memoria y los accesos vectorizados son más eficientes que los accesos de C++ con indirección de punteros.

   Con 400 000 iteraciones, Jacobi converge a 500×500. Las $\sim$ 167 000–207 000 iteraciones reales dejan un margen cómodo. Para mallas $N \geq 1\,000$ el método Jacobi se vuelve imprácticamente lento; se recomienda Gauss-Seidel ($\sim 50\,\%$ menos iteraciones) o SOR con parámetro de relajación óptimo $\omega^* \approx 2/(1 + \sin(\pi h))$.

   Los tres códigos son algorítmicamente equivalentes. Producen errores e iteraciones idénticas, confirmando la correcta implementación en los tres lenguajes. Las diferencias observadas son puramente de rendimiento, no de exactitud numérica.

Apéndice — Comandos de compilación y ejecución

Listing 4: Compilación y ejecución de los tres códigos.

```
# Fortran
gfortran -O2 -o poisson_laplace poisson_laplace.f90
./poisson_laplace 100 100   # malla 100x100
./poisson_laplace 500 500   # malla 500x500

# C++
g++ -O2 -o poisson_serial poisson_serial.cpp
./poisson_serial 100 100
./poisson_serial 500 500

# Python
python3 poisson_serial.py 100 100
python3 poisson_serial.py 500 500
```